

سوالات تحلیلی

سوال ۱: با مطالعه پروتکل سریال UART و بلوک مربوطه در STM32F401 پارامترهای قابل تنظیم آن را ذکر کنید.

- **Baud Rate:** symbols per second (baud).
- **Word length**
- **Parity**
- **Stop Bits:** number of bits sent after each data character to indicate the end of a transmission.

سوال ۲: نحوه فعال کردن وقفه دریافت داده های سریال UART را بیان کرده و رجیسترهای مربوطه را شرح دهید.

بعد از فعال کردن USART interrupt globally در NVIC کافیسست بیت RXNEIE را ست کنیم.

USART Control Register 1 (USART_CR1)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OVER8	Reserved	UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK
rw	Res.	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

- **UE:** USART Enable
- **RE:** Receiver Enable
- **TE:** Transmitter Enable
- **RXNEIE:** RXNE (Receive Data Register Not Empty) Interrupt Enable
- **TCIE:** Transmission Complete Interrupt Enable
- **TXEIE:** TXE (Transmit Data Register Empty) Interrupt Enable
- **M:** Word Length

سوال ۳: کنترل خطا در ارتباط سریال USART و SPI با چه منطقی انجام میگیرد؟ شرح دهید.

USART Status Register (USART_SR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CTS	LBD	TXE	TC	RXNE	IDLE	ORE	NF	FE	PE
						rc_w0	rc_w0	r	rc_w0	rc_w0	r	r	r	r	r

- TXE: Transmit data register empty
- TC: Transmission complete.
- RXNE: Read data register not empty
- IDLE: IDLE line detected
- ORE: Receive overrun. Received data has overwritten previous data in receive buffer
- NF: Noise flag. Receiver data bit samples don't agree.
- FE: Framing error. Received 0 for a stop bit, expected 1.
- PE: Parity error. Incorrect parity received.

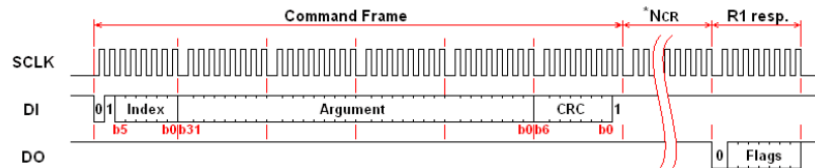
در USART خطا ها در داده ها به روش های Noise Error Flags و Parity Bits, Frame Error Flags همدل می شود.

SPI Control Register 2 (SPI_CR2)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								TXEIE	RXNEIE	ERRIE	FRF	Res.	SSOE	TXDMAEN	RXDMAEN
								rW	rW	rW	rW		rW	rW	rW

- TXEIE: Tx buffer empty interrupt enable
- RXNEIE: Rx buffer not empty interrupt enable
- ERRIE: Error interrupt enable
- FRF: Frame format
 - 0: SPI Motorola mode
 - 1: SPI TI mode
- SSOE: SS output enable
- TXDMAEN: Tx buffer DMA enable
- RXDMAEN: Rx buffer DMA enable

SPI Commands



- Host sends a six-byte command packet to card
 - Index, argument, CRC
- Host reads bytes from card until card signals it is ready
 - Card returns
 - 0xff while busy
 - 0x00 when ready without errors
 - 0x01-0x7f when error has occurred

در SPI، خطاها در حین conversation بین slave و master انجام می‌شود. و اگر خطایی وجود داشته باشد، command بخش سوم به عنوان ack خطا ارسال می‌شود.

سوال ۴: وظیفه کنترل داده‌ها و استخراج داده از بیت‌های کنترلی بر دوش برنامه‌نویس است یا ماژول سخت افزاری؟ شرح دهید که چرا از ماژول سخت افزاری برای این کار استفاده می‌شود؟

وظیفه کنترل آن این بیت‌های کنترلی از قبیل start bit، stop bit، و بیت‌های error detection و ... بر عهده ماژول سخت افزاری است. چرا که وارد شدن برنامه‌نویس در جزئیات آن باعث پیچیدگی بیشتر و همچنین خود موجب احتمال خطا و ایجاد overhead اضافی می‌شود. از طرفی اگر این بیت‌ها کنترل نشوند، نمی‌توان دیتایی دریافت/ارسال/هندل کرد، فلذا کنترل آن‌ها کاری لازم است که همواره باید انجام گردد.

گزارش دستور کار:

پیکربندی:

USART1 Mode and Configuration

Mode

Mode: Asynchronous ▼

Hardware Flow Control (RS232): Disable ▼

Configuration

Reset Configuration

✓ NVIC Settings
✓ DMA Settings
✓ GPIO Settings

✓ Parameter Settings
✓ User Constants

Configure the below parameters :

⏪ ⏩ ⓘ

▼ **Basic Parameters**

Baud Rate	9600 Bits/s
Word Length	8 Bits (including Parity)
Parity	None
Stop Bits	1

▼ **Advanced Parameters**

Data Direction	Receive and Transmit
Over Sampling	16 Samples

TIM2 Mode and Configuration

Mode

Slave Mode Disable ▼

Trigger Source Disable ▼

Clock Source Internal Clock ▼

Channel1 Disable ▼

Channel2 Disable ▼

Channel3 Disable ▼

Channel4 Disable ▼

Combined Channels Disable ▼

☐ Use ETR as Clearing Source

☐ XOR activation

☐ One Pulse Mode

Configuration

Reset Configuration

✔ User Constants

✔ NVIC Settings

✔ DMA Settings

✔ Parameter Settings

Configure the below parameters :

🔍 Search (Ctrl+F)

⏪

⏩

❗

▼

Counter Settings

Prescaler (PSC - 16 bits value) 16000-1

Counter Mode Up

Counter Period (AutoReload Reg... 1000-1

Internal Clock Division (CKD) No Division

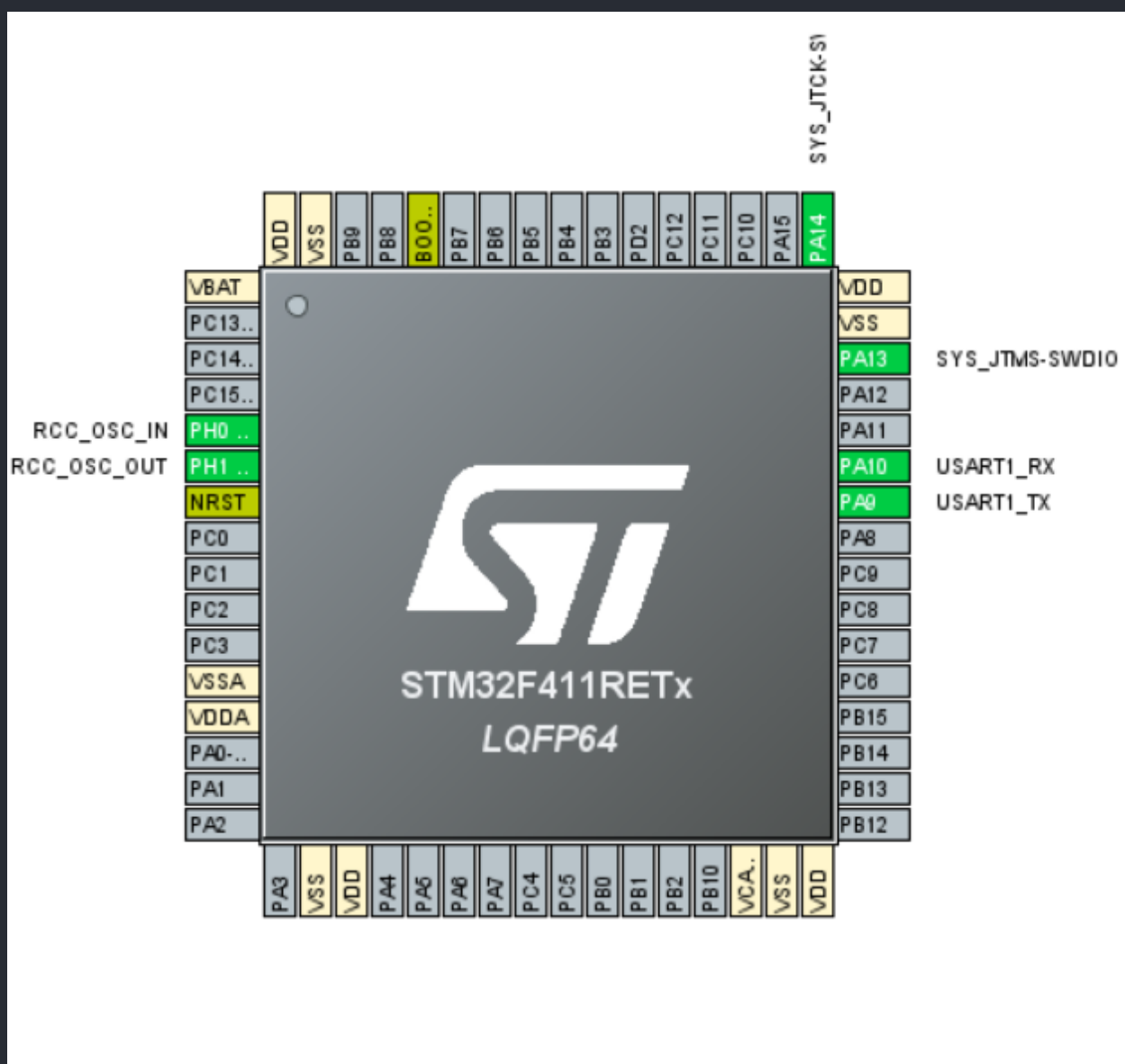
auto-reload preload Disable

▼

Trigger Output (TRGO) Parameters

Master/Slave Mode (MSM bit) Disable (Trigger input effect not delayed)

Trigger Event Selection Reset (UG bit from TIMx_EGR)



کد:

```

40  /* USER CODE END PM */
41
42  /* Private variables -----*/
43  TIM_HandleTypeDef htim2;
44
45  UART_HandleTypeDef huart1;
46
47  /* USER CODE BEGIN PV */
48  uint8_t Rx_data[10];
49  uint8_t write_buffer[2000];
50  volatile int write_buffer_tail = 0;
51  char N_str[10];
52  int N_str_tail = 0;
53  volatile int N = 0;
54  int row = 1;
55  /* USER CODE END PV */
56
57  /* Private function prototypes -----*/
58  void SystemClock_Config(void);
59  static void MX_GPIO_Init(void);
60  static void MX_TIM2_Init(void);
61  static void MX_USART1_UART_Init(void);
62  /* USER CODE BEGIN PFP */
63  int pascal(int row, int col);
64  int atoi(char* str);
65  void itoa(int num, char* str);
66  int intlen(int num);
67  void string_to_buffer(char* str);
68  void char_to_buffer(char c);
69  void indent(int num);
70  /* USER CODE END PFP */
71
72  /* Private user code -----*/
73  /* USER CODE BEGIN 0 */
74  void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
75  {
76      if (Rx_data[0] != '\r')
77      {
78          N_str[N_str_tail++] = Rx_data[0];
79          HAL_UART_Transmit_IT(&huart1, Rx_data, 1);
80          HAL_UART_Receive_IT(&huart1, Rx_data, 1);
81      }
82      else
83      {
84          N_str[N_str_tail] = '\0';
85          N_str_tail = 0;
86          N = atoi(N_str);
87
88          char_to_buffer('\r');
89          char_to_buffer('\n');
90          HAL_UART_Transmit_IT(&huart1, write_buffer, write_buffer_tail);
91      }
92  }
93
94  void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
95  {
96      write_buffer_tail = 0;
97  }

```

```

main.c
99 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
100 {
101     if (htim->Instance == TIM2)
102     {
103         if (row <= N)
104         {
105             indent(N - row);
106
107             for (int i = 1; i <= row; i++)
108             {
109                 int num = pascal(row, i);
110
111                 char str[10];
112                 itoa(num, str);
113
114                 string_to_buffer(str);
115                 char_to_buffer(' ');
116             }
117             char_to_buffer('\r');
118             char_to_buffer('\n');
119
120             HAL_UART_Transmit_IT(&huart1, write_buffer, write_buffer_tail);
121
122             if (++row > N)
123             {
124                 char_to_buffer('\r');
125                 char_to_buffer('\n');
126                 char str[] = "Enter N: \0";
127                 string_to_buffer(str);
128                 HAL_UART_Transmit_IT(&huart1, write_buffer, write_buffer_tail);
129
130                 HAL_UART_Receive_IT (&huart1, Rx_data, 1);
131
132                 row = 1;
133                 N = 0;
134             }
135         }
136     }
137 }
138

```



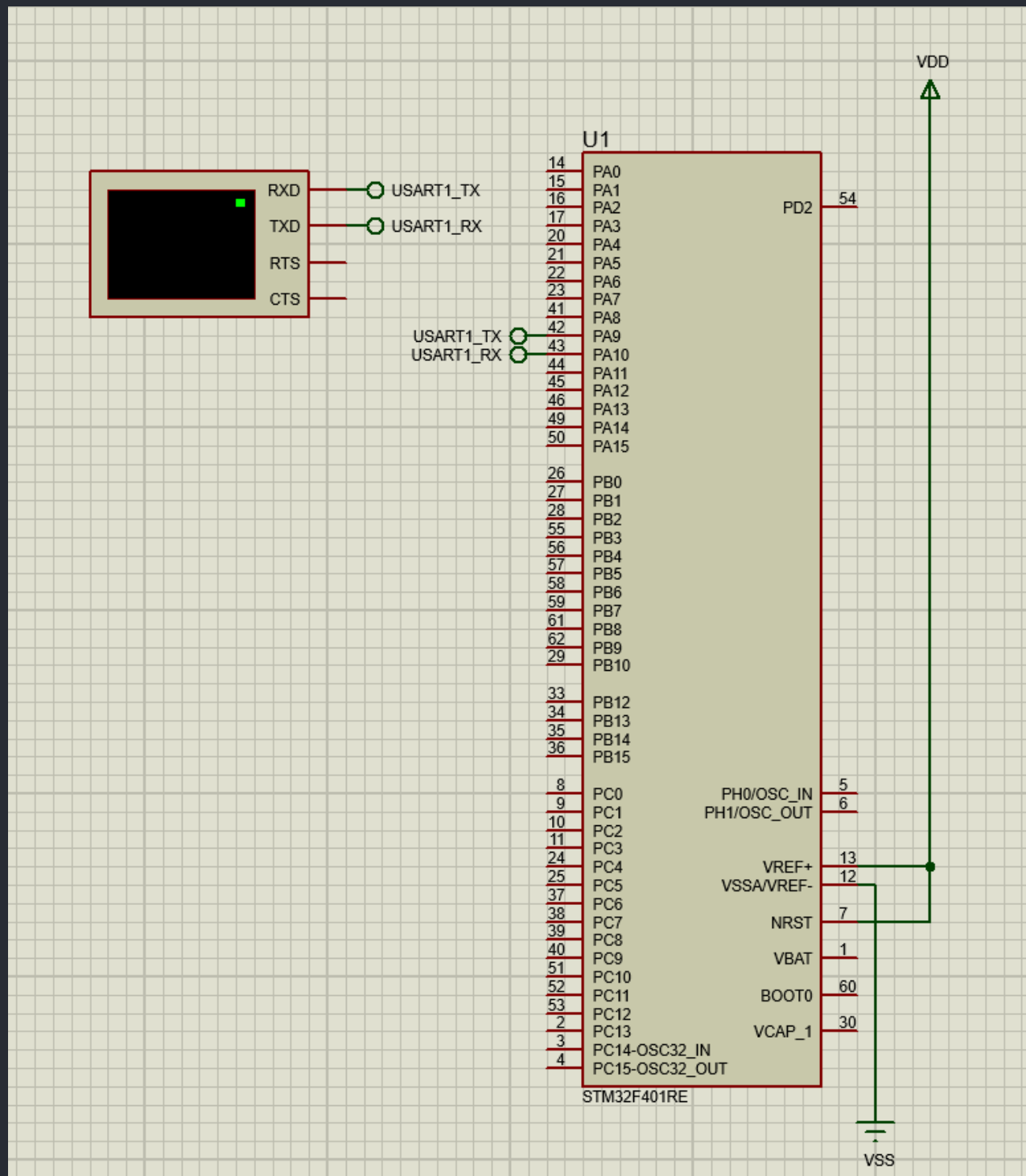
```

main.c
139 int pascal(int row, int col)
140 {
141     if (col == 1 || row == col)
142         return 1;
143     else
144         return pascal(row - 1, col) + pascal(row - 1, col - 1);
145 }
146
147 int atoi(char* str)
148 {
149     int res = 0;
150     for (int i = 0; str[i] != '\0'; ++i)
151         res = res * 10 + str[i] - '0';
152     return res;
153 }
154
155 void itoa(int num, char* str)
156 {
157     if (num == 0)
158     {
159         str[0] = '0';
160         str[1] = '\0';
161         return;
162     }
163
164     int i = intlen(num);
165     str[i--] = '\0';
166
167     while (num != 0)
168     {
169         int rem = num % 10;
170         str[i--] = rem + '0';
171         num = num/10;
172     }
173 }
174
175 int intlen(int num)
176 {
177     int len = 0;
178     while (num != 0)
179     {
180         num = num/10;
181     }
182 }
183
184 void string_to_buffer(char* str)
185 {
186     for (int i = 0; str[i] != '\0'; i++)
187     {
188         write_buffer[write_buffer_tail++] = str[i];
189     }
190 }
191
192 void char_to_buffer(char c)
193 {
194     write_buffer[write_buffer_tail++] = c;
195 }
196
197 void indent(int num)
198 {
199     for (int i = 0; i < num; i++)
200     {
201         write_buffer[write_buffer_tail++] = ' ';
202     }
203 }
204
205 /* USER CODE END 0 */
206
207
208

```

```
main.c
210  * @brief The application entry point.
211  * @retval int
212  */
213  int main(void)
214  {
215      /* USER CODE BEGIN 1 */
216
217      /* USER CODE END 1 */
218
219      /* MCU Configuration-----*/
220
221      /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
222      HAL_Init();
223
224      /* USER CODE BEGIN Init */
225
226      /* USER CODE END Init */
227
228      /* Configure the system clock */
229      SystemClock_Config();
230
231      /* USER CODE BEGIN SysInit */
232
233      /* USER CODE END SysInit */
234
235      /* Initialize all configured peripherals */
236      MX_GPIO_Init();
237      MX_TIM2_Init();
238      MX_USART1_UART_Init();
239      /* USER CODE BEGIN 2 */
240
241      char str[] = "Enter N: \0";
242      string_to_buffer(str);
243      HAL_UART_Transmit_IT(&huart1, write_buffer, write_buffer_tail);
244
245      HAL_UART_Receive_IT (&huart1, Rx_data, 1);
246      HAL_TIM_Base_Start_IT(&htim2);
247
248      /* USER CODE END 2 */
249  }
```

تست:



```
Virtual Terminal
Enter N: 7
      1
     1 1
    1 2 1
   1 3 3 1
  1 4 6 4 1
 1 5 10 10 5 1
1 6 15 20 15 6 1
12
      1
     1 1
    1 2 1
   1 3 3 1
  1 4 6 4 1
 1 5 10 10 5 1
1 6 15 20 15 6 1
1 7 21 35 35 21 7 1
1 8 28 56 70 56 28 8 1
1 9 36 84 126 126 84 36 9 1
1 10 45 120 210 252 210 120 45 10 1
1 11 55 165 330 462 462 330 165 55 11 1
|
```